***** CONFIDENTIAL to Bluespec, Inc. and Select Partners *****

Porting Catamaran to a new Host+FPGA platform

Bluespec, Inc.

Document Rev. 2023-03-22.0
Original author Rishiyur S. Nikhil

Note: “Catamaran” used to be called “AWSteria” during early development. Please only use the name “Catamaran” from now on.

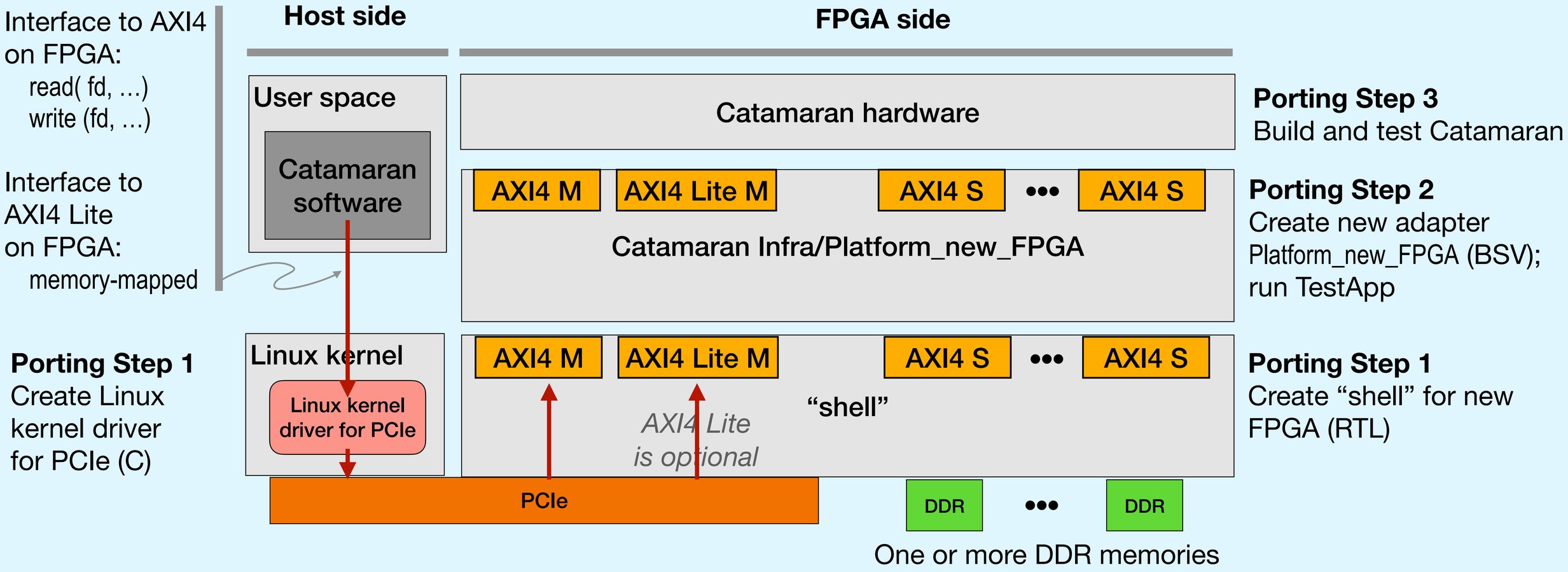
Porting Catamaran to a new Host+FPGA Platform

Involves 3 steps:

1. Create an FPGA-side “shell” and Host-side device driver
   - This can be done completely independently of Bluespec, Inc. and involves no code from Bluespec, Inc.
2. In Bluespec’s “Catamaran_Infra/” distribution, create a new “Platform_New_FPGA/” adapter, similar to the existing adapters “Platform_AWSF1/” and “Platform_VCU118/”
   - Use Catamaran_Infra/TestApp/ to ensure this is working
3. Build and test Catamaran on top of (2) and (1).
   - This should require no “porting”, just “building”

Porting Catamaran to a new Host+FPGA Platform

Pictorial view



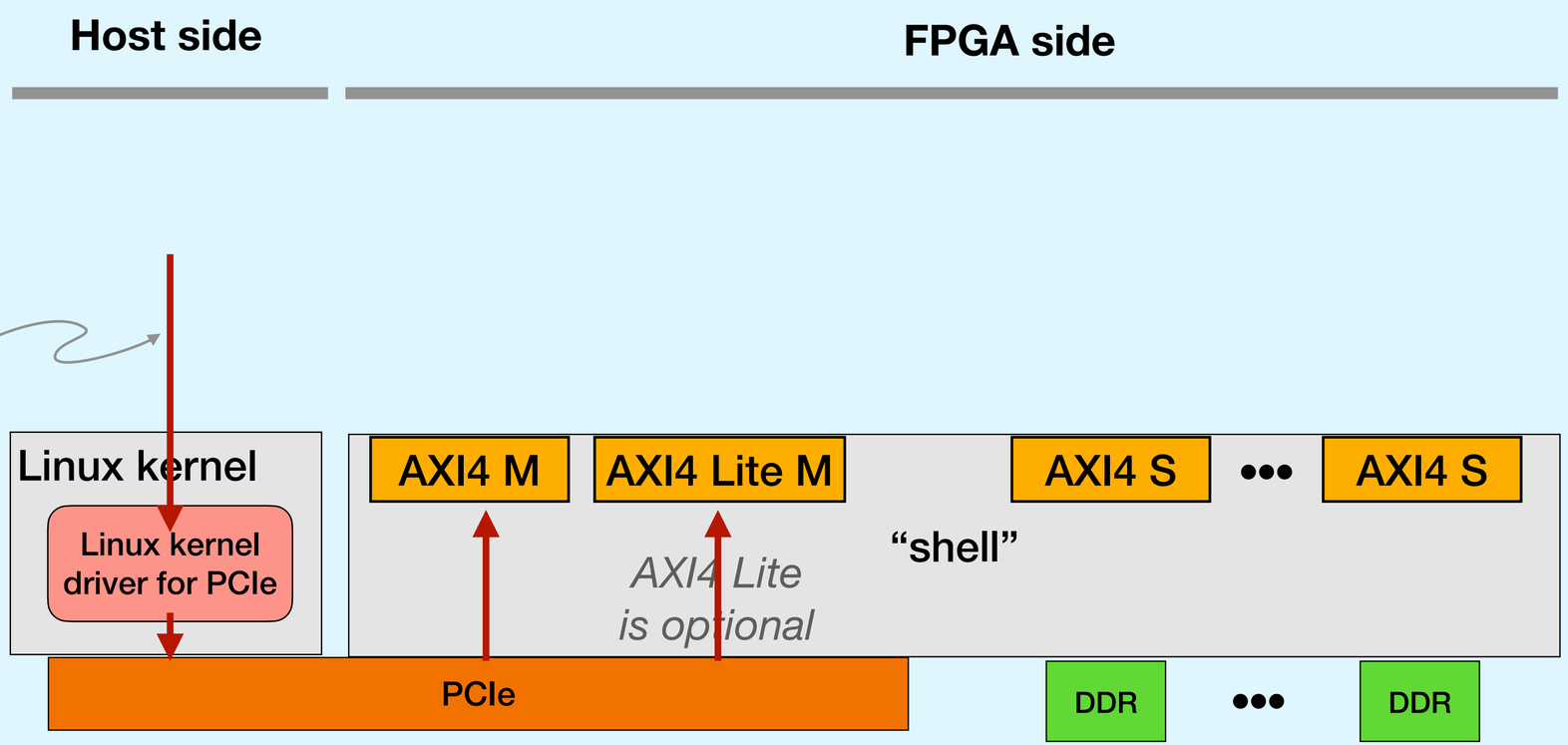
1. Create an FPGA-side “shell” and Host-side device driver

Porting Catamaran to a new Host+FPGA Platform

Step 1 does not involve any Bluespec hardware or software

Interface to AXI4 on FPGA:
read(fd, …)
write (fd, …)

Interface to AXI4 Lite on FPGA:
memory-mapped



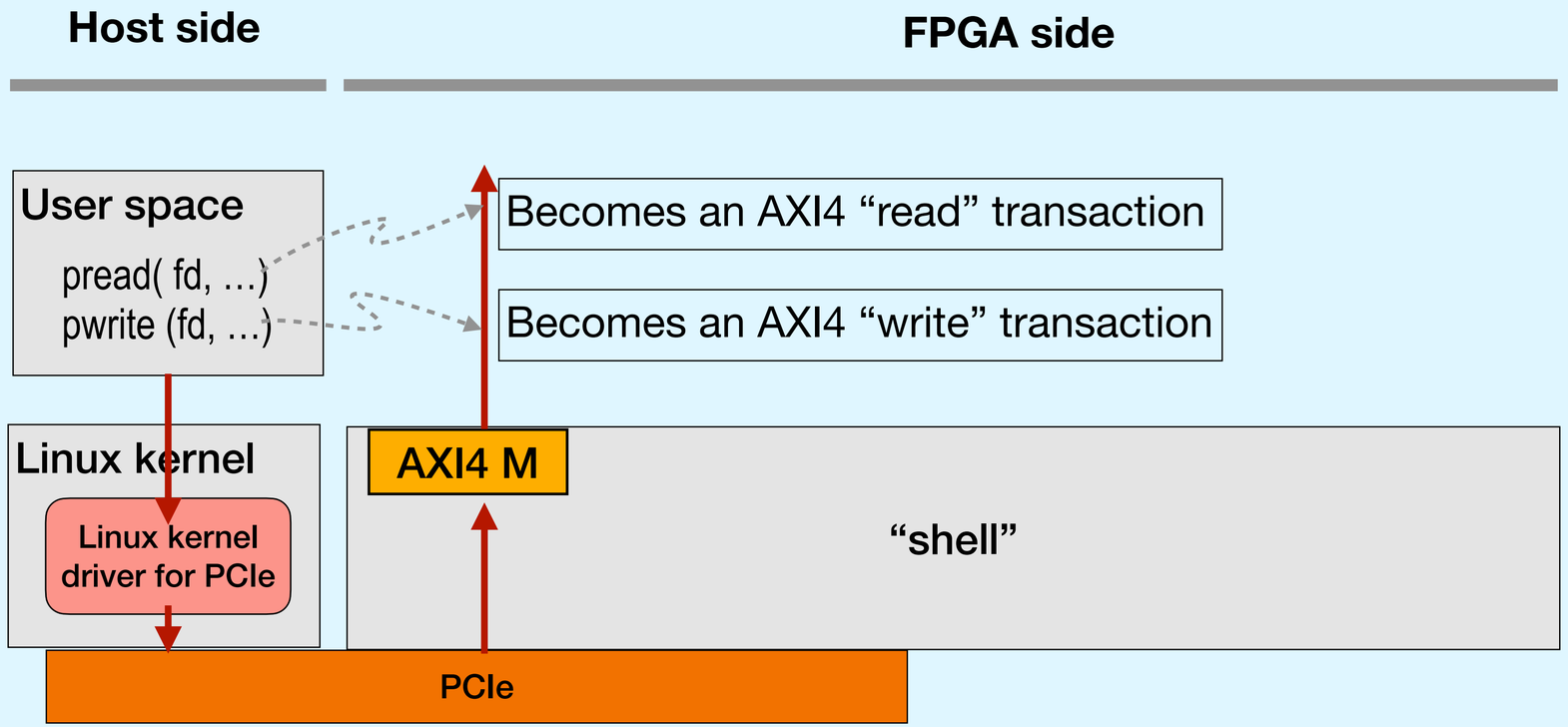
Porting Step 1
Create Linux kernel driver for PCIe (C)

Porting Step 1
Create "shell" for new FPGA (RTL)

One or more DDR memories

Porting Catamaran to a new Host+FPGA Platform

Step 1: Goal for AXI4 M(master/manager) on PCIe



Notes:

- *AXI4: 64b addr, 512b data*
- *May need “bursts”*
- *May need “lane alignment”, “byte enables”*

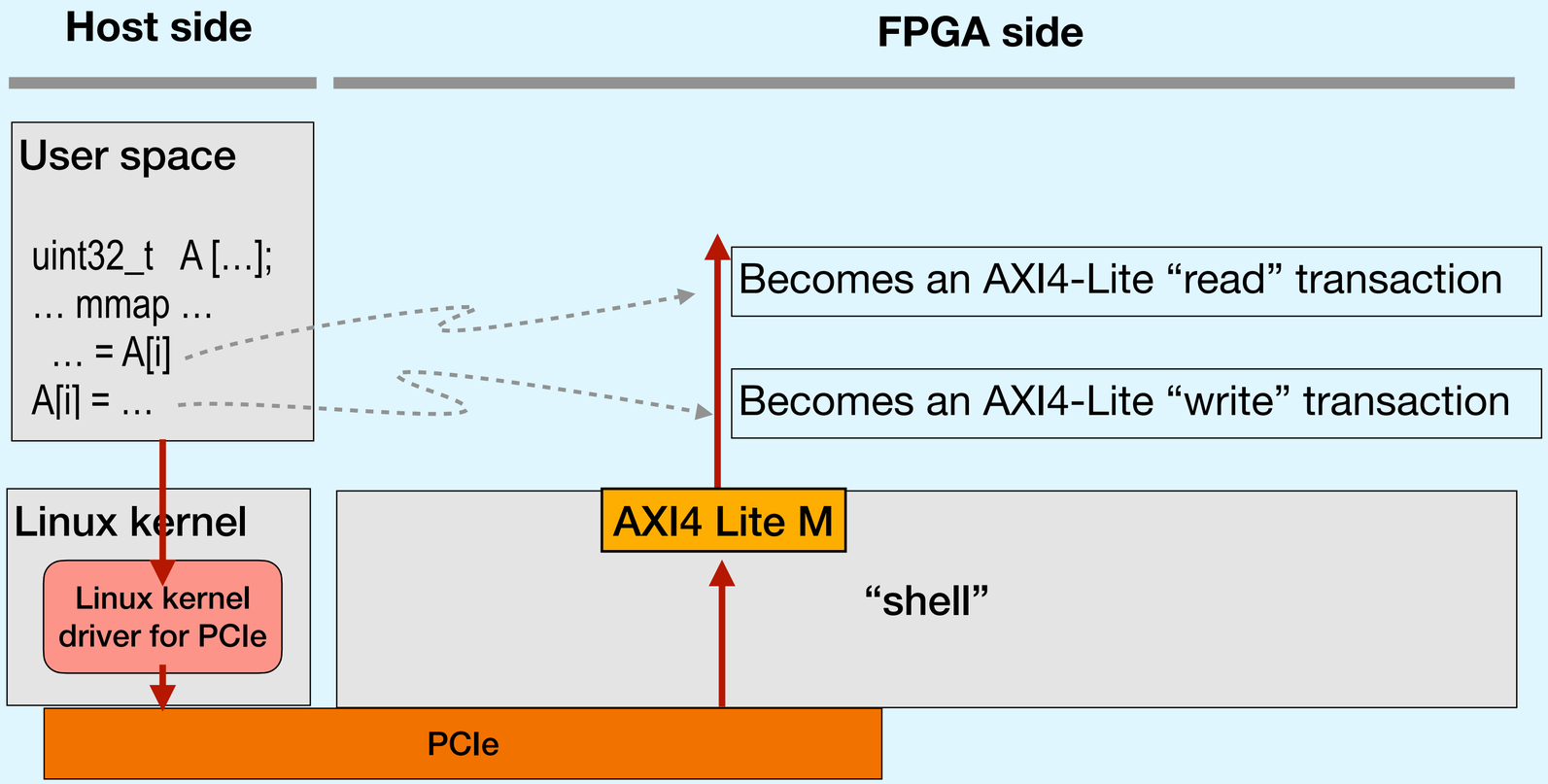
From Unix ‘man’ pages:

ssize_t pread (int fildes, void *buf, size_t nbyte, off_t offset);

ssize_t pwrite (int fildes, const void *buf, size_t nbyte, off_t offset);

Porting Catamaran to a new Host+FPGA Platform

Step 1: Goal for AXI4-Lite M (master/manager) on PCIe



Notes:

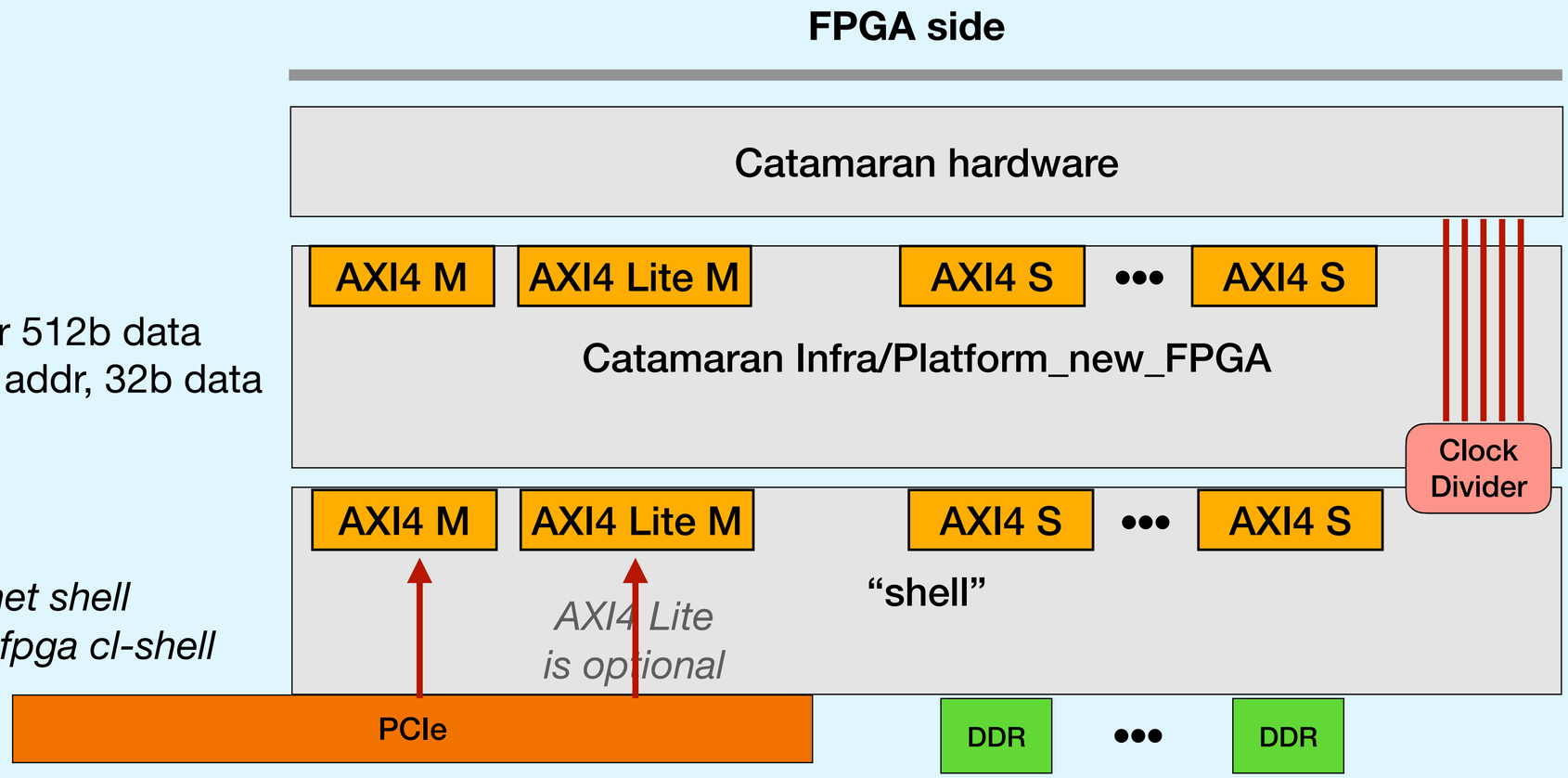
- *AXI4-Lite: 32b addr, 32b data*
- *AXI4-Lite: 32b address space is “memory mapped” into host-side user space*
- *The AXI4-Lite interface is optional, or implementation can be postponed; Catamaran can work with just the AXI4 interface.*

Porting Catamaran to a new Host+FPGA Platform

Step 1: FPGA-side “shell”:

AXI4: 64b addr 512b data
AXI4 Lite: 32b addr, 32b data

VCU118: Garnet shell
AWS F1: aws-fpga cl-shell



One or more DDR memories

VCU118: 2 x 2GB
AWS F1: 4 x 16GB

Clocks:
All of Catamaran (other than the Core/CPU) runs on one clock:
VCU118: 250 MHz
AWSF1: 250 MHz

We supply 5 additional slower clocks to Catamaran (since some Cores/CPUs cannot run at 250 MHz).

The clock divider can be in Catamaran_Infra (e.g, as in Platform_VCU118) or can be in the “shell” (e.g., as in Platform_AWSF1)

Porting Catamaran to a new Host+FPGA Platform

Step 1: Example shells (open-source) for existing FPGA platforms

- VCU118: Garnet shell: https://github.com/CTSRD-CHERI/garnet
  - This file: garnet/include/partition_ports.vh
    shows the Verilog port-list for the Garnet shell
- Amazon AWS F1: https://github.com/aws/aws-fpga
  - This file: aws-fpga/hdk/common/shell_stable/design/interfaces/cl_ports.vh
    shows the Verilog port-list for the aws-fpga shell
    (it also has many extra ports which can be ignored)

Porting Catamaran to a new Host+FPGA Platform

Step 1: Examples Host-side kernel driver (free, open-source)

- Existing host-side kernel driver for PCIe, from Xilinx:
  - XDMA Linux kernel driver:
    https://support.xilinx.com/s/article/65444?language=en_US
    https://github.com/Xilinx/dma_ip_drivers

2. In Bluespec's "Catamaran_Infra/" distribution, create a new "Platform_New_FPGA/" adapter

Create new “Platform_New_FPGA/” adapter

- In code delivered by Bluespec, Inc., see “Catamaran_Infra/”

  (Note: may be called “BlueCloudV_Infra/” in your distribution)

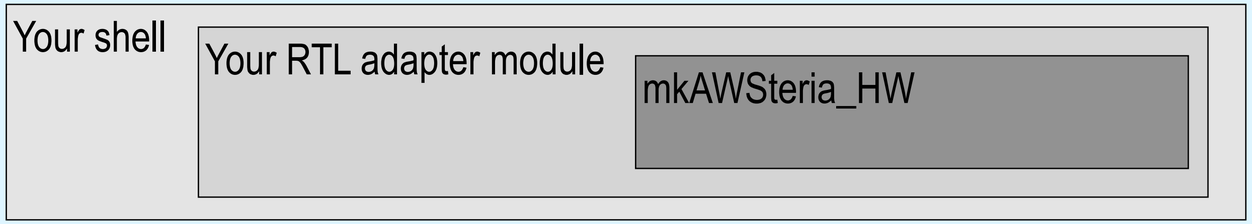
- It contains two existing platform adapters:
  - “Platform_VCU118/” and “Platform_AWSF1/”
- Goal: create new “Platform_YourFPGA/” for your FPGA board.
- “TestApp/” contains a small test application to make sure Catamaran_Infra works
  - Ensure that it builds with Platform_YourFPGA and runs on your FPGA

Create new “Platform_New_FPGA/” adapter

- We suggest your new “Platform_YourFPGA/” have the following sub-directories:

```
Platform_YourFPGA
├── HW
└── Host
```

- In “HW/”, you should implement an RTL adapter module
  - That is instantiated by your shell
  - That instantiates mkAWSteria_HW (which will be provided by Catamaran)



- The interface for mkAWSteria_HW can be seen in:
  BSV: Catamaran_Infra/Include_API/AWSteria_HW_IFC.bsv
  Verilog: Catamaran_Infra/Include_API/AWSteria_HW_Empty.v

Create new “Platform_New_FPGA/” adapter

- We suggest your new “Platform_YourFPGA/” have the following sub-directories:

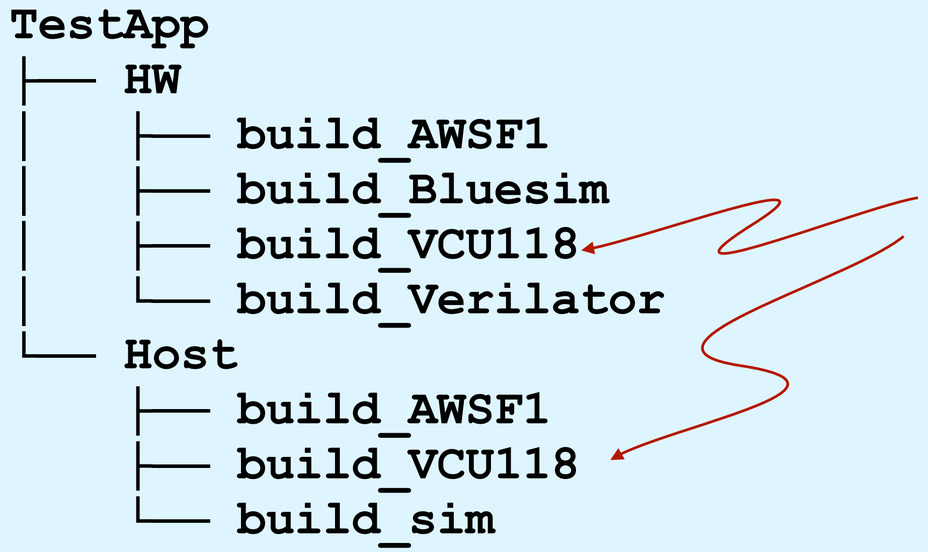
```
Platform_YourFPGA
├── HW
└── Host
```

- In “Host/”, you should implement “AWSteria_Host_lib.c” which interacts with your kernel device driver

- The interface for AWSteria_Host_lib.c can be seen in:
  C: Catamaran_Infra/Include_API/AWSteria_Host_lib.h

Create new “Platform_New_FPGA/” adapter

- To test “Platform_YourFPGA/”, update “TestApp/”:



Add and populate “build” directories for HW-side and Host-side, similar to these

- Load the HW-side bitfile into your FPGA
- Run the Host-side TestApp software

END